BigQuery AI.GENERATE

システム規模のレコード評価

全レコードにGeminiで評価・ラベリング

こんなニーズありませんか？

システムから流れてくる大量のレコード

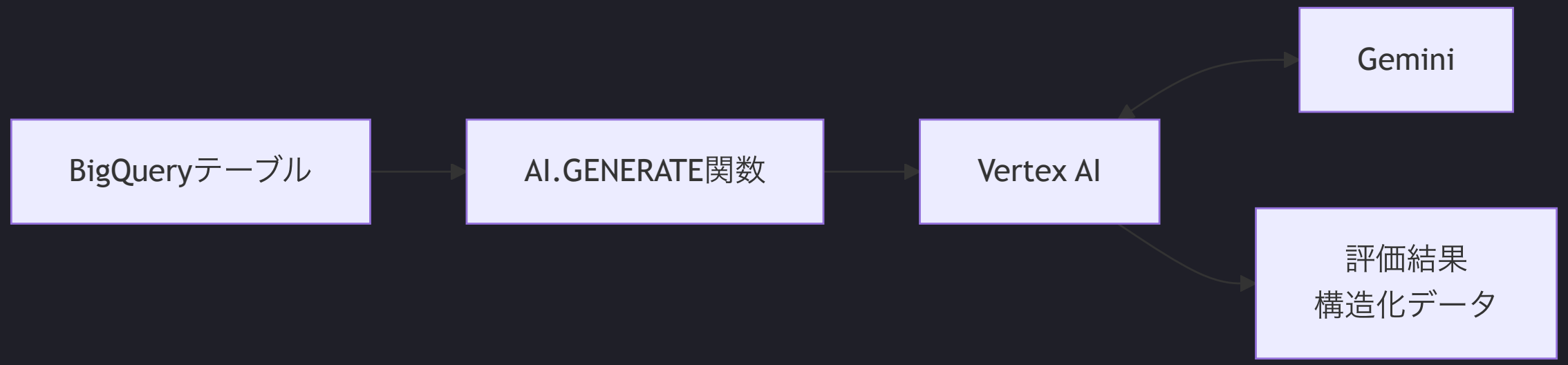
- カスタマーレビュー → **感情分析したい**
- システムログ → **異常検知スコアを付けたい**
- テキストデータ → **自動分類・ラベリングしたい**

従来の課題

- 外部API呼び出しは管理が複雑
- バッチ処理の実装が大変
- データガバナンスの懸念

BigQueryからGeminiを呼び出す

アーキテクチャ



3

GENERATE関数ファミリーの全種類

テキスト生成

関数名	処理単位	特徴
AI.GENERATE	Row-wise	output_schema指定可（推奨）
AI.GENERATE_TEXT	Table-valued	テーブル全体処理
AI.GENERATE_TABLE	Row-wise	構造化データ生成
ML.GENERATE_TEXT	Table-valued	旧式（非推奨）

その他の生成AI関数

- **AI.GENERATE_EMBEDDING**: ベクトル埋め込み生成
- **AI.CLASSIFY**: カテゴリ分類
- **AI.SCORE**: スコアリング
- **AI.IF**: 条件フィルタリング

選択基準

- Row-wise処理（1レコードずつ） → **AI.GENERATE**
- 構造化された結果 → **AI.GENERATE**（output_schema）

4

output_schemaの威力

構造化された評価結果を得る

```
SELECT
  record_id,
  AI.GENERATE(
    'このレビューを評価してください',
    review_text,
    STRUCT(
      'sentiment STRING, score INT64,
       category ARRAY<STRING>'
      AS output_schema
    )
  ) AS evaluation
FROM customer_reviews
```

結果: sentiment="positive", score=85, category=["product"]

5

実践例1: レビューの自動評価

```
CREATE OR REPLACE TABLE analyzed_reviews AS
SELECT
  review_id,
  review_text,
  AI.GENERATE(
    'このレビューの感情を分析してください',
    review_text,
    STRUCT(
      'sentiment STRING, confidence_score FLOAT64,
       key_points ARRAY<STRING>' AS output_schema,
      'gemini-2.5-flash' AS model
    )
  ).result.* -- 構造化結果を展開
FROM customer_reviews
WHERE analyzed_at IS NULL; -- 未分析のみ
```

ポイント: 1レコードごとにGeminiが評価、構造化された結果を取得

6

実践例2: ログデータの異常検知

```
SELECT
  log_id,
  log_message,
  AI.GENERATE(
    'このログは正常/警告/エラーのどれですか？',
    log_message,
    STRUCT(
      'category STRING, severity INT64,
       requires_action BOOL' AS output_schema
    )
  ).result AS classification
FROM system_logs
WHERE created_at > TIMESTAMP_SUB(
  CURRENT_TIMESTAMP(), INTERVAL 1 HOUR
)
```

結果: category="error", severity=8, requires_action=true

7

dbtでの増分更新パターン

課題: 既に評価済みのレコードは再実行したくない

```
{{
  config(
    materialized='incremental',
    unique_key='record_id'
  )
}}

SELECT
  record_id,
  created_at,
  AI.GENERATE(...) AS evaluation
FROM {{ source('raw', 'records') }}

{% if is_incremental() %}
  WHERE record_id NOT IN (SELECT record_id FROM {{ this }})
{% endif %}
```

効果: コスト削減、システム組み込みに最適

8

管理型AI関数との組み合わせ

さらに高度な評価も可能

- **AI.CLASSIFY**: ユーザー定義カテゴリに分類
- **AI.SCORE**: 品質や類似度でランク付け
- **AI.IF**: 自然言語条件でフィルタリング

```
SELECT
  record_id,
  AI.SCORE('品質スコア', description) AS quality,
  AI.CLASSIFY(['重要', '通常', '低優先'], content) AS priority
FROM records
```

ポイント: プロンプト最適化済み、カスタマイズ不要で使える

9

システム組み込みのベストプラクティス

コスト管理

- 増分更新で再実行を防ぐ
- WHERE句で対象を絞る
- Provisioned Throughputで安定化

品質管理

- output_schemaで構造化
- confidenceスコアで精度確認
- 定期的なプロンプト見直し

運用管理

- dbtでパイプライン化
- ステージング→評価→検証の3段階
- エラーハンドリング（status列をチェック）

10

他手法との比較

手法	データ移動	型指定	システム組み込み
BigQuery AI.GENERATE	不要	◎	◎
Cloud Functions + API	必要	△	○
Dataflow + Vertex AI	必要	○	○
外部バッチ処理	必要	△	△

選択ガイド

- データ基盤がBigQuery → **AI.GENERATE**
- 複雑なカスタム処理 → Cloud Functions
- リアルタイム処理 → Vertex AI直接

11

まとめ

システム規模の評価・分類の時代

メリット:

- SQLだけで全レコード評価可能
- データ移動なしでセキュア
- 型指定で後続処理が容易
- dbtで運用効率化

次のステップ:
テキスト生成・画像生成の次は **レコード評価**

BigQuery AI.GENERATE、使ってみませんか？

12

参考リンク

公式ドキュメント

- AI.GENERATE function
- AI.GENERATE_TEXT function
- Generative AI overview

ブログ記事

- New BigQuery gen AI functions
- SQL reimagined for the AI era

13